INTERNAL ENGINEERING DOC

How Kite Works

A complete technical reference for the data pipeline, caching architecture, API layer, RAG system, and frontend. Written for engineers joining the project or anyone who wants to understand how the gears turn.

Architecture Overview

Kite is a three-tier application. A **React/Vite frontend** talks to a **Python/FastAPI backend** deployed on Railway. All persistent state lives in **Supabase** (PostgreSQL + pgvector). There is no Redis, no message queue, and no separate worker process — the scheduler runs inside the FastAPI process using APScheduler.

FRONTEND

React 18, Vite, React Router v6. Deployed to **Vercel**. All API calls go through `frontend/src/api/client.js` — no component calls `fetch()` directly.

BACKEND

FastAPI + Uvicorn on **Railway**. One process, async HTTP handlers, APScheduler for background jobs. No worker separation at MVP scale.

DATABASE

Supabase (hosted PostgreSQL). Used for quote cache, time series cache, SEC embeddings (pgvector), and structured financials. Two connection modes: REST client and raw psycopg2.

EXTERNAL APIS

Twelve Data — live quotes, OHLCV time series, symbol search. **EDGAR/SEC** — 10-K/10-Q text and XBRL financial facts. **Anthropic Claude** — RAG answer generation.

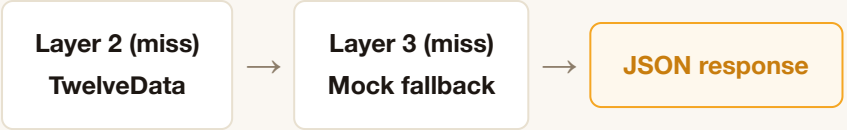
ℹ The frontend never touches Supabase or external APIs directly. Everything goes through the FastAPI backend. This keeps API keys server-side and allows the backend to apply rate limiting, caching, and auth logic in one place.

Full Data Flow

Quote Request (most common path)

When a page loads and needs prices for a list of tickers:

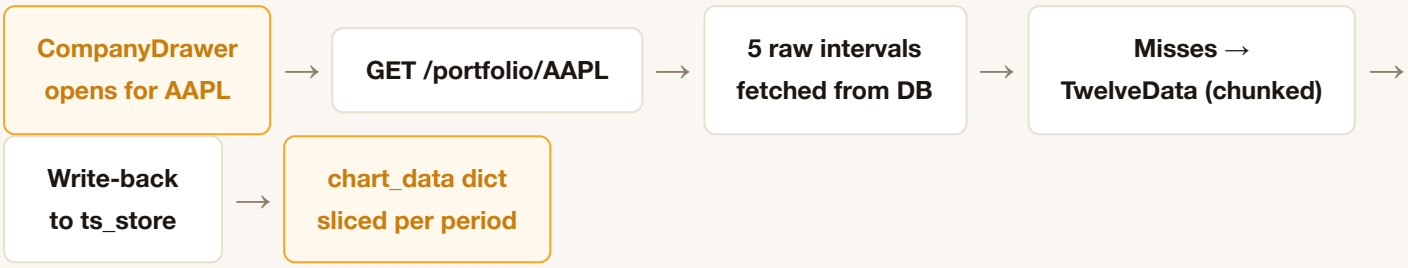
User opens page → `getQuotes(tickers)` client.js → GET /quotes FastAPI → Layer 1 Supabase DB →



In normal operation the scheduler pre-warms the DB every 15 minutes, so **Layer 1 hits for ~95% of requests** — zero TwelveData credits consumed for routine loads.

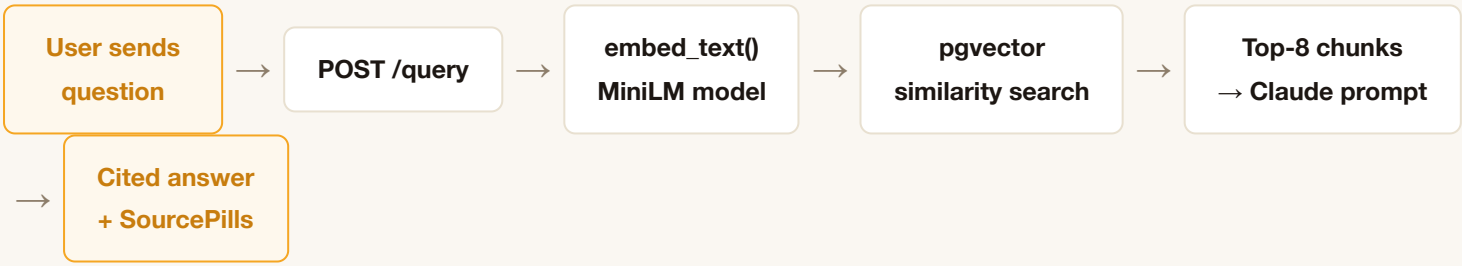
Chart Data Request (on CompanyDrawer open)

When a user clicks a stock to open the CompanyDrawer, full time series are needed:



The 5 raw series (5min, 1h, 1day, 1week, 1month) are fetched once and sliced to build all 11 chart periods (1D, 1W, 1M, 3M, 6M, YTD, 1Y, 2Y, 5Y, 10Y, MAX). After the first load the DB has all 5 series cached for 6 hours.

AI Query Flow



Environment Variables

VARIABLE	USED BY	DESCRIPTION
`TWELVE_DATA_API_KEY`	Backend	TwelveData API key. Free tier: 800 calls/day, 8/min.
`ANTHROPIC_API_KEY`	Backend	Claude API key. Used by reasoner.py for RAG answers.
`SUPABASE_URL`	Backend	Supabase project URL (e.g. `https://xyz.supabase.co`). Used by the REST client.
`SUPABASE_KEY`	Backend	Supabase anon/service key. Used by the REST client.
`DATABASE_URL`	Backend	Direct Postgres connection string. Used by psycopg2 for pgvector and bulk writes.
`VITE_API_URL`	Frontend	Backend base URL. Dev: `http://localhost:8000`. Prod: Railway URL.

⚠ `DATABASE_URL` and `SUPABASE_DB_URL` both work for psycopg2 connections — the code checks both. Get the connection string from Supabase → Settings → Database → URI mode. Include the password in the string; do not put it separately.

API Endpoints

All routes are defined in `backend/main.py`. The FastAPI app starts with `uvicorn backend.main:app --reload --port 8000`. Interactive docs are at `http://localhost:8000/docs`.

GET **`/health`** Railway uptime monitor ping — Live

Returns `{"status":"ok"}`. No DB calls. Railway pings this every 30 seconds.

GET **`/quotes?tickers=AAPL,MSFT,NVDA`** Batch quote (DB-first) — Live

Primary quote endpoint. DB → TwelveData API → mock fallback. Returns price, change %, sparkline stub, and all KPI fields. Used by every page that shows a stock row.

- Layer 1: reads `market_quotes` table for rows < 15 min old
- Layer 2: calls TwelveData `/quote` for any miss; writes back to DB
- Layer 3: returns from `_MOCK_PORTFOLIO` for common tickers if both layers fail
- Layer 4: empty stub (price=0) for unknown tickers so drawer still opens

GET **`/portfolio/{ticker}`** Full quote + all chart series — Live

Called only when a CompanyDrawer opens. Fetches the 5 raw time series (5min/1h/1day/1week/1month) from DB, falls back to TwelveData for misses, writes back, then slices into the `chart_data` dict keyed by period (1D, 1W, … MAX). Also fetches a live quote with KPIs.

- 5 raw intervals → 11 chart periods; only 5–6 TwelveData calls on first load
- Subsequent loads within 6 hours: 0 TwelveData calls (all served from `time_series_cache`)

GET **`/portfolio_history?tickers=AAPL,MSFT&period=1Y`** Multi-ticker daily close series — Live

Returns `{"AAPL": {"closes": [...], "dates": [...]}, "SPY": {...}}`. SPY is always appended as benchmark. Used by the Portfolios page performance chart. Backed by the same `time_series_cache` table and 1day interval.

GET **`/financials/{ticker}`** Quarterly + annual income statement — Live

Returns `quarterly` and `annual` arrays with revenue, gross profit, EBITDA, net income. Pulls from Twelve Data's income_statement endpoint. Used by CompanyDrawer Financials and Trends tabs.

GET **/search?q=apple** Symbol search proxy — Live

Proxies TwelveData `/symbol_search`. API key stays server-side. Returns up to 8 results: symbol, name, exchange, type, country. Called by the global search bar with 250ms debounce.

POST **/query** RAG AI answer with citations — Live

Body: `{"question": "...", "tickers": ["AAPL"]}`. Embeds the question → pgvector similarity search (top 8 chunks) → Claude generates cited answer. Returns `answer`, `sources`, `sourced`.

- If no chunks found: returns "I don't have a source for that." — Claude is never called
- If chunks exist but API key missing: raises 500 with clear error message

POST **/ingest** Full SEC filing ingestion pipeline — Live

Body: `{"tickers": ["AAPL"], "filing_types": ["10-K", "10-Q"]}`. Runs the full 5-stage pipeline: EDGAR download → chunking → embedding → pgvector upsert → XBRL financials. Can take minutes per ticker. Called manually or from admin UI.

POST **/ingest-financials** Fast financial-only ingestion (no embeddings) — Live

XBRL data only — ~5–10s per ticker. Use this to populate the Financials/Trends tabs without running the full embedding pipeline.

GET **/news** · **/briefing/{date}** News feed and daily briefing — Partial

News uses `ingestion/news.py` which is wired up; briefing returns empty items (stub). Both endpoints exist so the frontend can be built without waiting for backend implementation.

Rate Limiter

Twelve Data's free tier allows **8 API credits per minute**. Each ticker in a `/quotes` batch counts as 1 credit; each ticker in a time series batch also counts as 1 credit. The rate limiter in `backend/ingestion/market.py` enforces a global cap across concurrent requests.

Implementation

A thread-safe sliding window using a deque of timestamps:

```python
_RL_MAX = 7       # stay 1 under the limit (8) for headroom
_RL_WIN = 61.0    # 61s window (slightly over 60 to be safe)

def _api_call(fn, credits: int = 1):
    effective = min(credits, _RL_MAX)    # clamp for large batches
    while True:
        with _rl_lock:
            now = time.time()
            while _rl_times and _rl_times[0] < now - _RL_WIN:
                _rl_times.popleft()              # expire old entries
            if len(_rl_times) + effective <= _RL_MAX:
                t = time.time()
                for _ in range(effective):
                    _rl_times.append(t)          # register these credits
                break
            wait = (_rl_times[0] + _RL_WIN) - now if _rl_times else 0
        if wait > 0:
            time.sleep(wait)             # sleep OUTSIDE lock
    return fn()
```

Key design decisions

- **Sleep outside the lock** — other threads can check the deque while this one waits, instead of blocking each other.
- **Clamp** `effective = min(credits, _RL_MAX)` — a 21-ticker batch would try to reserve 21 credits, but since only 7 fit, we track 7. Without this, `len(times) + 21 <= 7` is never true and the loop spins forever.
- **Guard** `if _rl_times else 0` — if the deque is empty (no recent calls), wait is 0 and we proceed immediately.

Chunked batching

For time series fetches with many tickers, `get_batch_time_series` splits the ticker list into chunks of `_RL_MAX` (7) and calls the rate limiter between each chunk:

```python
for i in range(0, len(missing), _RL_MAX):
    chunk = missing[i : i + _RL_MAX]
    result = _api_call(
        lambda: _fetch_time_series_batch(chunk, interval, outputsize),
        credits=len(chunk),
    )
```

Caching Layers

Kite has four caching layers, innermost to outermost:

LAYER	LOCATION	TTL	WHAT'S CACHED
1 — In-memory quote cache	`_quote_cache` dict in market.py	15 min	Single-ticker `/quote` responses. Cleared on server restart.
2 — In-memory time series cache	`_ts_batch_cache` dict in market.py	1 hour	Raw time series for a (ticker, interval, outputsize) key. Cleared on restart.
3 — DB quote cache	`market_quotes` table in Supabase	15 min (staleness check)	Quotes for the full MARKET_UNIVERSE (~50 tickers). Warmed every 15 min by scheduler.
4 — DB time series cache	`time_series_cache` table in Supabase	6 hours	OHLCV close series for (ticker, interval, outputsize). Survives server restarts.

✦ Layers 3 and 4 are the important ones for production — they survive server restarts and are shared across all server instances. Layers 1 and 2 are fast in-process shortcuts that reduce DB reads during a single server session.

Cache miss flow (quotes)

1. Check `market_quotes` in Supabase → return if fresh
2. Call TwelveData `/quote` for missing tickers (respects rate limiter)
3. Write fresh rows back to `market_quotes` (upsert on ticker PK)
4. Fall back to `_MOCK_PORTFOLIO` if API also fails

Cache miss flow (time series)

1. Check `time_series_cache` for each of the 5 raw intervals
2. For any misses: call TwelveData `/time_series` in chunks of 7
3. Write results back to `time_series_cache` (upsert on ticker+interval+outputsize)
4. Slice the 5 raw series into the `chart_data` dict keyed by period label

Scheduler

The scheduler runs inside the FastAPI process using `APScheduler (AsyncIOScheduler)`. It is started in the FastAPI `lifespan` context and shut down cleanly on server exit.

Jobs

JOB ID	TRIGGER	WHAT IT DOES
`market_quote_refresh`	Every 15 min	Fetches the latest quote for all ~50 tickers in `MARKET_UNIVERSE` + any user-ingested tickers from the `financials` table. Upserts to `market_quotes`. **Skips if market is closed** (saves API credits outside trading hours).
`nightly_financials`	2:00 AM UTC daily	Re-fetches EDGAR XBRL financial facts for every ticker in the `financials` table and overwrites stale rows. Keeps the Financials tab data fresh without manual re-ingestion.

Market hours detection

```
def _is_market_open() -> bool:
    now = datetime.now(timezone.utc)
    if now.weekday() >= 5:          # Sat=5, Sun=6
        return False
    h = now.hour + now.minute / 60
    return 13.5 <= h <= 21.0        # 9:30am-4pm ET in UTC
```

> ℹ The scheduler does not account for US market holidays. On bank holidays during the week it will still attempt a refresh, but TwelveData will return the previous close values — which is fine since prices haven't changed.

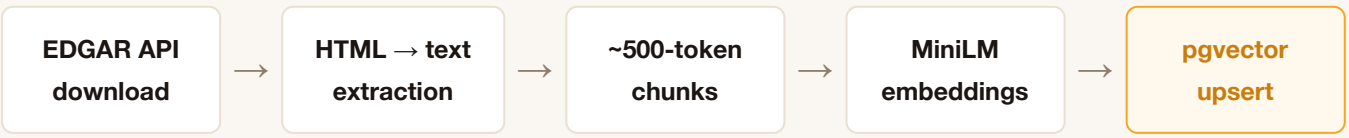
RAG Pipeline (AI Assistant)

The AI assistant answers questions by retrieving relevant passages from SEC filings stored in pgvector, then passing them to Claude for reasoning. This is Retrieval-Augmented Generation (RAG).

Embedding model

`all-MiniLM-L6-v2` (Sentence Transformers) — a small, fast model that runs locally. Produces 384-dimensional dense vectors. The same model is used at both ingestion time (chunk embedding) and query time (question embedding), which is required for cosine similarity to be meaningful.

Stage 1: Ingestion (`POST /ingest`)



- `edgar.py` — downloads 10-K and 10-Q filings from SEC EDGAR, strips HTML to plain text
- `chunker.py` — splits text into ~500-token chunks with 50-token overlap to preserve context boundaries
- `embedder.py` — calls `all-MiniLM-L6-v2` via `sentence_transformers`; normalizes vectors to unit length
- `store.py` — upserts chunks into the `documents` pgvector table with ON CONFLICT on `(ticker, doc_type, fiscal_year, chunk_index)`

Stage 2: Query (`POST /query`)

- `retriever.py` — embeds the question, calls `similarity_search()` in pgvector with optional ticker filter, returns top-8 chunks sorted by cosine similarity
- `reasoner.py` — formats chunks into a numbered context block, calls Claude, parses the cited answer into a `QueryResponse`

```
# reasoner.py — prompt structure
user_message = (
    "Here are the relevant source passages:\n\n"
    f"{context}\n\n"          # numbered [1]…[8] chunks
    f"Question: {question}"
)
```

Model selection

MODEL	SPEED	COST/QUERY	USE CASE
`claude-haiku-4-5`	~1–2s	~$0.001	Current default. Good for factual lookups from chunks.
`claude-sonnet-4-6`	~3–5s	~$0.006	Recommended upgrade. Better synthesis and reasoning.
`claude-opus-4-7`	~8–12s	~$0.04	Overkill for RAG. Not recommended.

✦ Enable **prompt caching** on the `KITE_SYSTEM_PROMPT` (it's long and static) to cut input token costs by ~90% on repeated queries. Pass `cache_control: {"type": "ephemeral"}` on the system message block.

Ingestion Pipeline Detail

SEC EDGAR (`ingestion/edgar.py`)

Uses the EDGAR full-text search API and the filing index to download 10-K/10-Q HTML filings. Strips HTML tags, normalizes whitespace, and preserves page structure. No EDGAR API key required (public).

XBRL Financial Facts (`ingestion/xbrl.py`)

Pulls structured financial data from `data.sec.gov/api/xbrl/companyfacts`. Extracts revenue, gross profit, EBITDA, net income, and EPS for both annual and quarterly periods. No parsing of filing text — pure structured data. Used by `/ingest-financials` and the nightly scheduler job.

News (`ingestion/news.py`)

Wired up to the `/news` endpoint. Fetches headlines filtered by ticker list and filter type (portfolio, trending, top, recent). Currently returns real data when the underlying source is available.

Supabase Schema

All tables live in the default `public` schema in Supabase. Run these SQL statements in Supabase → SQL Editor to create them.

documents (pgvector — SEC filing chunks)

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS documents (
    id            BIGSERIAL    PRIMARY KEY,
    ticker        TEXT         NOT NULL,
    doc_type      TEXT         NOT NULL,   -- '10-K', '10-Q', etc.
    fiscal_year   TEXT,
    source_label  TEXT,
    source_url    TEXT,
    page_number   INTEGER,
    chunk_index   INTEGER      NOT NULL,
    content       TEXT         NOT NULL,
    embedding     vector(384),            -- MiniLM-L6-v2 output dimension
    metadata      JSONB        DEFAULT '{}',
    created_at    TIMESTAMPTZ  DEFAULT NOW(),
    UNIQUE (ticker, doc_type, fiscal_year, chunk_index)
);

-- Index for fast cosine similarity search
CREATE INDEX documents_embedding_idx
    ON documents USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100);
```

market_quotes (quote cache)

```sql
CREATE TABLE IF NOT EXISTS market_quotes (
    ticker        TEXT         PRIMARY KEY,
    name          TEXT,
    price         FLOAT,
    change_pct    FLOAT,
    pe_ratio      FLOAT,
    market_cap    FLOAT,
    eps           FLOAT,
    beta          FLOAT,
    week_52_high  FLOAT,
    week_52_low   FLOAT,
    volume        BIGINT,
    fetched_at    TIMESTAMPTZ  DEFAULT NOW()
);
```

time_series_cache (chart data)

```sql
CREATE TABLE IF NOT EXISTS time_series_cache (
    ticker        TEXT         NOT NULL,
    interval      TEXT         NOT NULL,   -- '5min', '1h', '1day', '1week', '1month'
    outputsize    INTEGER      NOT NULL,   -- number of bars requested
    data          JSONB        NOT NULL DEFAULT '{}',
    fetched_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    PRIMARY KEY (ticker, interval, outputsize)
);
```

financials (XBRL structured data)

```sql
CREATE TABLE IF NOT EXISTS financials (
    id            BIGSERIAL    PRIMARY KEY,
    ticker        TEXT         NOT NULL,
    period_type   TEXT         NOT NULL,   -- 'annual' | 'quarterly'
    fiscal_year   TEXT,
    fiscal_quarter TEXT,
    revenue       FLOAT,
    gross_profit  FLOAT,
    ebitda        FLOAT,
    net_income    FLOAT,
    eps           FLOAT,
    created_at    TIMESTAMPTZ  DEFAULT NOW(),
```

```
    UNIQUE (ticker, period_type, fiscal_year, fiscal_quarter)
);
```

Data freshness summary

TABLE	UPDATED BY	FRESHNESS
`market_quotes`	Scheduler every 15 min + API miss write-back	≤15 min during market hours
`time_series_cache`	API miss write-back (on first CompanyDrawer open)	≤6 hours
`financials`	Nightly scheduler at 2 AM UTC + manual /ingest	≤24 hours
`documents`	Manual /ingest only	On-demand (SEC filings rarely change)

DB Connections

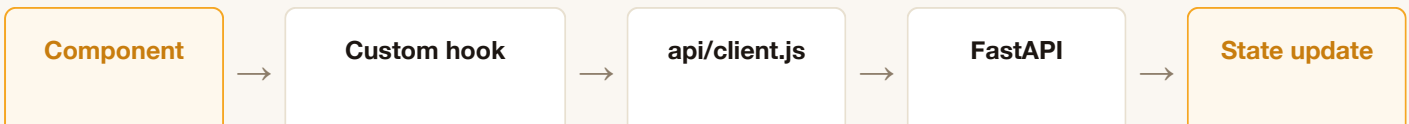
Two connection modes are used for different purposes:

MODE	MODULE	WHEN USED
Supabase REST client `get_supabase_client()`	`backend/db.py`	General queries via Supabase's auto-generated REST API. Requires `SUPABASE_URL` + `SUPABASE_KEY`.
psycopg2 direct `get_pg_conn()`	`backend/db.py`	pgvector operations (embedding upsert, cosine similarity search), bulk writes for cache tables. Requires `DATABASE_URL`. The REST client does not support the `vector` data type.

⚠ psycopg2 connections are opened per-request and closed immediately (context manager). Supabase free tier allows ~20 concurrent connections. Do not hold connections open across async boundaries — this will exhaust the pool under load. Connection pooling (PgBouncer) is a roadmap item.

Frontend Data Flow

The frontend follows a strict pattern: **components never call the API directly**. All network calls go through `frontend/src/api/client.js`, which exports named async functions. Components either call these directly or use them through custom hooks.



mounts | (useHoldings etc.) | function | backend | → re-render

Global state

Two pieces of global state are held in `App.jsx`'s `Layout` component so they persist across page navigation:

- `drawerStock` — the stock currently displayed in `CompanyDrawer`. Set by search results, assistant action chips, or any StockRow click.
- `assistantOpen` — whether the `AssistantPanel` is visible. Toggled by the "Ask Kite" button in the nav.

Both are lifted to `Layout` so the drawer and panel persist when the user navigates between pages (e.g. opens a company from the News page and navigates to Portfolios without losing the drawer).

Custom Hooks

HOOK	FILE	WHAT IT DOES
`useHoldings()`	`hooks/useHoldings.js`	Reads the user's portfolio holdings from localStorage. Returns `{holdings, addHolding, removeHolding}`. Holdings are `[{ticker, shares, avgCost}]`. No API calls — pure local state.
`useWatchlist()`	`hooks/useWatchlist.js`	Reads/writes the watchlist from localStorage. Returns `{watchlist, addToWatchlist, removeFromWatchlist}`.
`usePortfolios()`	`hooks/usePortfolios.js`	Manages multiple named portfolios in localStorage. Returns the active portfolio and CRUD operations.
`useTheme()`	`hooks/useTheme.js`	Reads the user's theme preference (light/dark) from localStorage and applies CSS variables to `:root`.

> ℹ All user data (holdings, watchlist, portfolios, theme) is stored client-side in `localStorage` at MVP. There is no user account system, no server-side profile storage, and no auth. This is a known limitation — user data is browser-specific and lost if the browser storage is cleared.

Pages & Key Components

PAGE / COMPONENT	FILE	KEY DATA CALLS
Dashboard	`pages/Dashboard.jsx`	`getQuotes(holdingTickers)` for snapshot + P&L math; `getNews()` for feed; watchlist from hook

Portfolios	`pages/Portfolios.jsx`	`getQuotes()` for table; `getPortfolioHistory()` for performance chart
Watchlist	`pages/Watchlist.jsx`	`getQuotes(watchlistTickers)`
Market	`pages/Market.jsx`	`getQuotes(indexETFs + sectorETFs + movers)`
News	`pages/News.jsx`	`getNews({filter})`
Screener	`pages/Screener.jsx`	Filter UI only — results endpoint not yet wired
Profile	`pages/Profile.jsx`	localStorage reads — no API calls
CompanyDrawer	`components/CompanyDrawer.jsx`	`getPortfolioFull(ticker)` (on open), `getFinancials(ticker)`
AssistantPanel	`components/AssistantPanel.jsx`	`queryKite(question, portfolioTickers)`
GlobalSearch	`App.jsx`	`searchSymbols(query)` debounced 250ms; `getQuotes([symbol])` on select

AssistantPanel — action detection

After every assistant response, `detectActions(text, portfolioTickers)` scans the answer text for:

- **Ticker mentions** — regex `/\b([A-Z]{1,5})\b/g` matched against a set of ~50 common large-caps plus the user's portfolio tickers. Matched tickers become clickable chips that open the CompanyDrawer.
- **Navigation cues** — regex patterns for words like "market", "screener", "watchlist", "portfolio", "news". Matched patterns surface a navigation chip (e.g. "Open Market →") that calls `useNavigate()`.

Infrastructure & Deployment

COMPONENT	PLATFORM	HOW DEPLOYED
Frontend (React/Vite)	Vercel	Git push to main → auto-deploy. `VITE_API_URL` set in Vercel env.
Backend (FastAPI)	Railway	Dockerfile or Nixpacks auto-detect. Start command: `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`
Database	Supabase	Managed PostgreSQL + pgvector. Connection strings in Railway env vars.

Local development

```
# Terminal 1 — Backend
cd kite
```

```
pip install -r requirements.txt
uvicorn backend.main:app --reload --port 8000

# Terminal 2 — Frontend
cd frontend
npm install
npm run dev              # Vite starts on http://localhost:5173

# .env (project root)
TWELVE_DATA_API_KEY=...
ANTHROPIC_API_KEY=...
SUPABASE_URL=...
SUPABASE_KEY=...
DATABASE_URL=...

# frontend/.env.local
VITE_API_URL=http://localhost:8000
```

CORS

Currently set to `allow_origins=["*"]` in `main.py` for development convenience. Before production go-live, restrict this to the Vercel deployment URL and any preview URL patterns.

API Costs & Credit Math

API	FREE TIER	WHEN CREDITS ARE CONSUMED
TwelveData	800 credits/day · 8/min	1 credit per ticker per API call. The scheduler uses ~50 credits every 15 min during market hours (≈ 2,400/day = 3× the free limit). Upgrade to Growth tier ($29/mo) for production.
Anthropic Claude	No free tier	Each /query call: ~2K input tokens + 8 chunk context + system prompt ≈ $0.001–$0.006. With prompt caching on the system prompt: ~$0.0003/query after first call.
Supabase	500 MB DB, 2 GB bandwidth	Free tier covers MVP. pgvector index + embeddings for ~10 tickers × 2 filings × 200 chunks = ~4,000 rows × 384 floats ≈ 6 MB. Well within free tier.

⚠ The scheduler refreshing 50 tickers every 15 minutes during market hours (6.5h/day) consumes **~1,300 TwelveData credits/day**. This exceeds the free 800/day limit. Either reduce MARKET_UNIVERSE or upgrade the plan before enabling the scheduler in production.

Technical Roadmap

ITEM	PRIORITY	NOTES
Screener endpoint — filter stocks by KPIs	High	Read from `market_quotes` + `financials` tables. No new API calls needed.
Supabase user auth	High	Replace localStorage holdings/watchlist with server-side tables per user.
Upgrade Claude to Sonnet for /query	Medium	Change `DEFAULT_MODEL` in reasoner.py; add prompt caching on system message.
TwelveData plan upgrade	Medium	Free tier (800/day) too low once scheduler is active during market hours.
Connection pooling (PgBouncer)	Medium	psycopg2 opens a new connection per request; fine at MVP but will exhaust Supabase connection limit at scale.
Briefing generation (`/briefing`)	Medium	Wire up `outputs/briefing.py` — summarize alerts, P&L, news into a daily brief.
Move /ingest to background task	Low	Currently synchronous; can block server for minutes. Move to APScheduler background job with status polling.
Restrict CORS origins	Low (pre-launch)	Change `allow_origins=["*"]` to specific Vercel URLs.
News data source	Low	Integrate a real-time financial news API (e.g. Polygon.io news, NewsAPI finance).